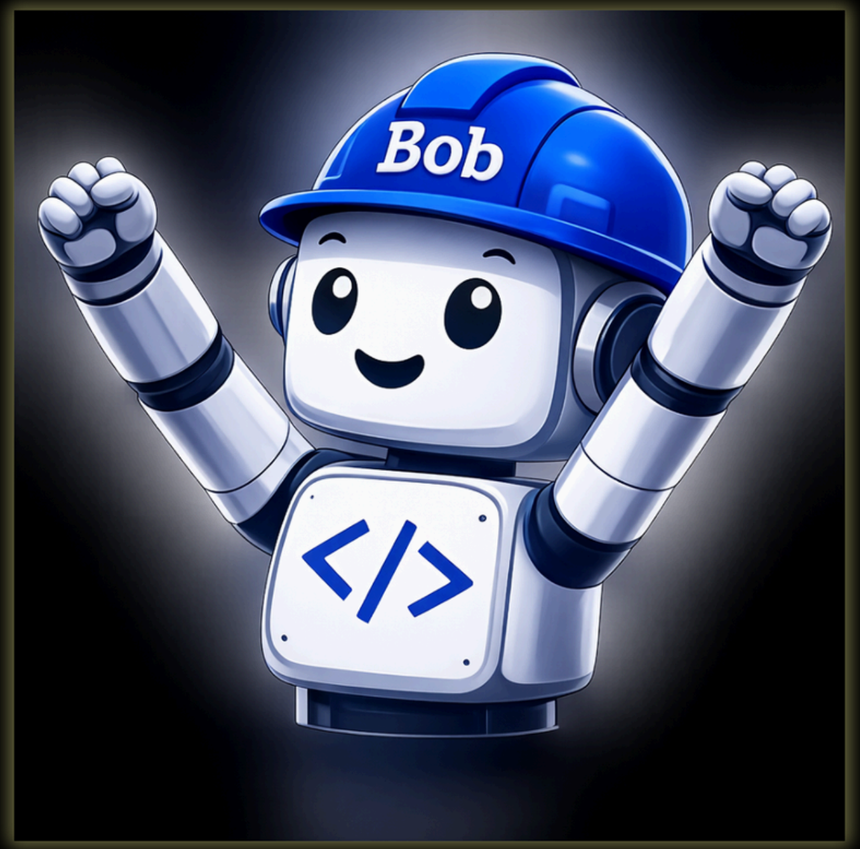


IBM Bob

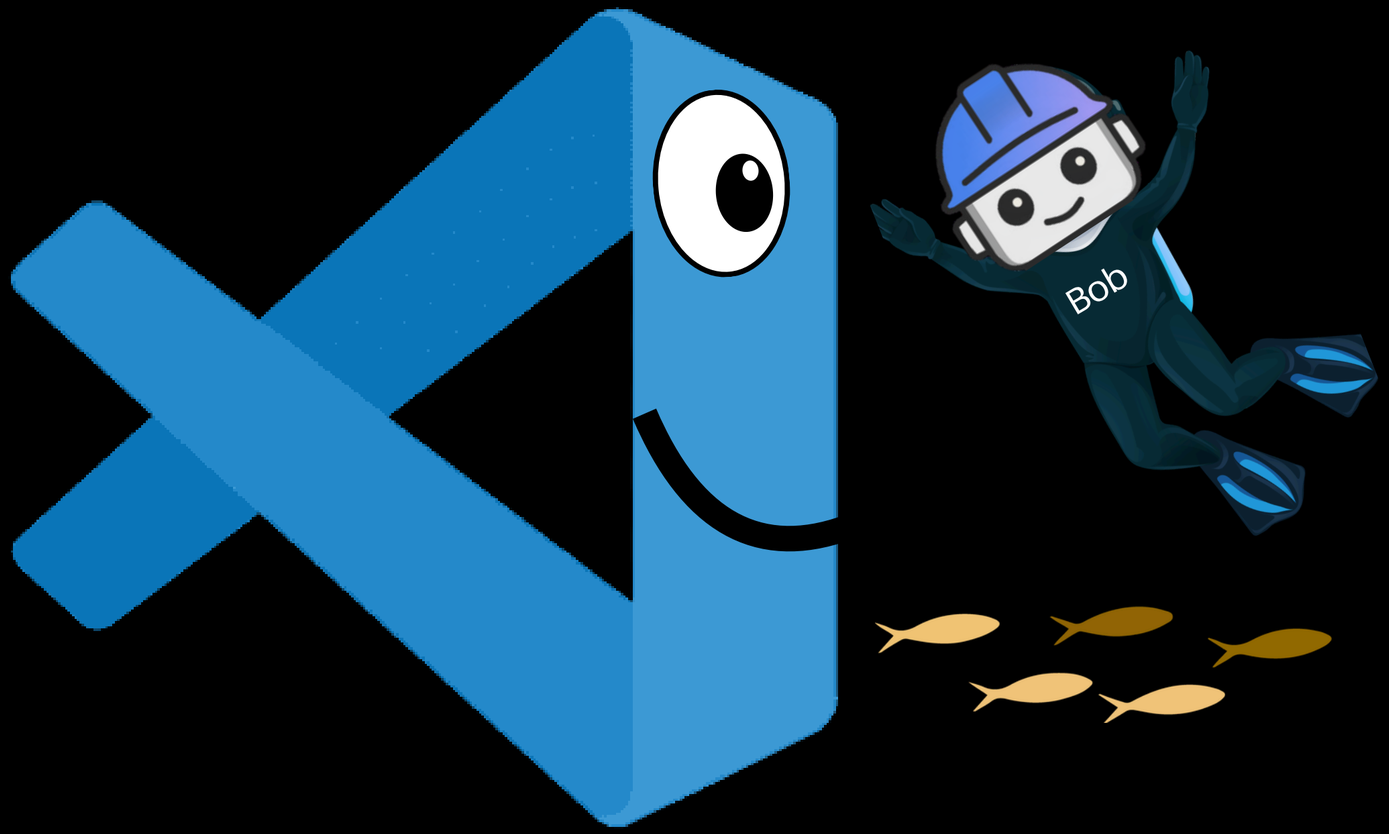
Developer Guide

Main functions and use cases



Neal Whittle

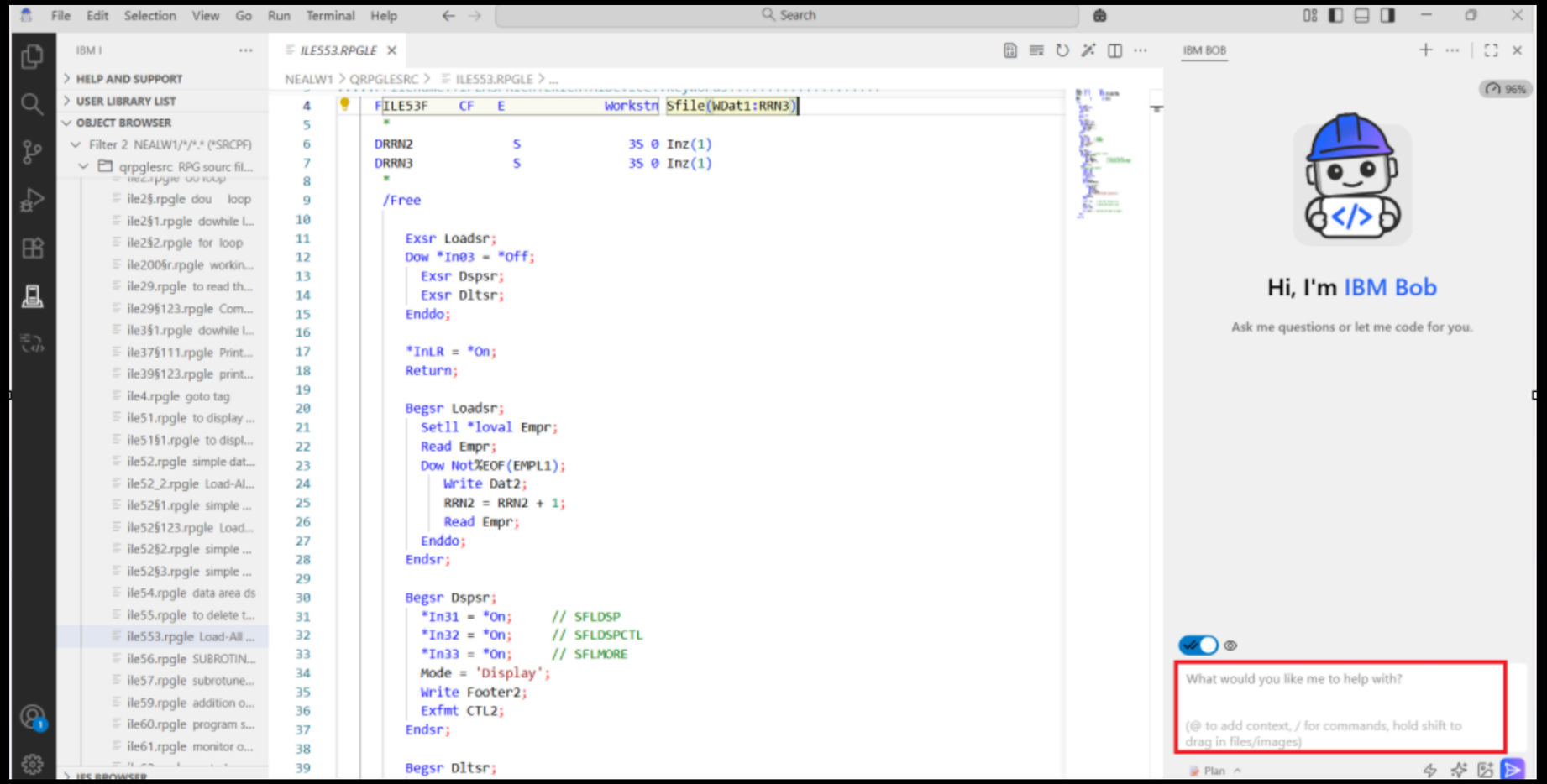
Bob is a fork of VS Code



VS Code was released in 2015 by Microsoft as a free, open-source code editor and has established itself as the ‘big fish’ in the IDE space, consistently ranking first in developer surveys.

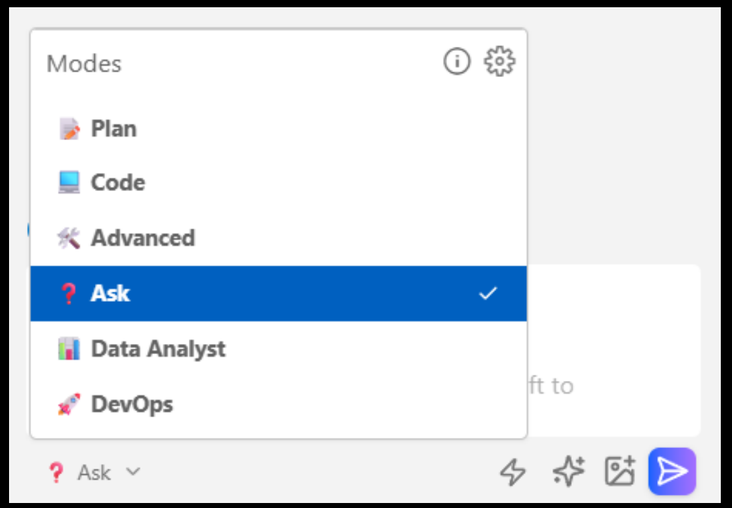
Developers already familiar with VS Code can make a easy transition to Bob, as preferences and settings can be retained.

Users of VS Code will immediately recognise a familiar layout.



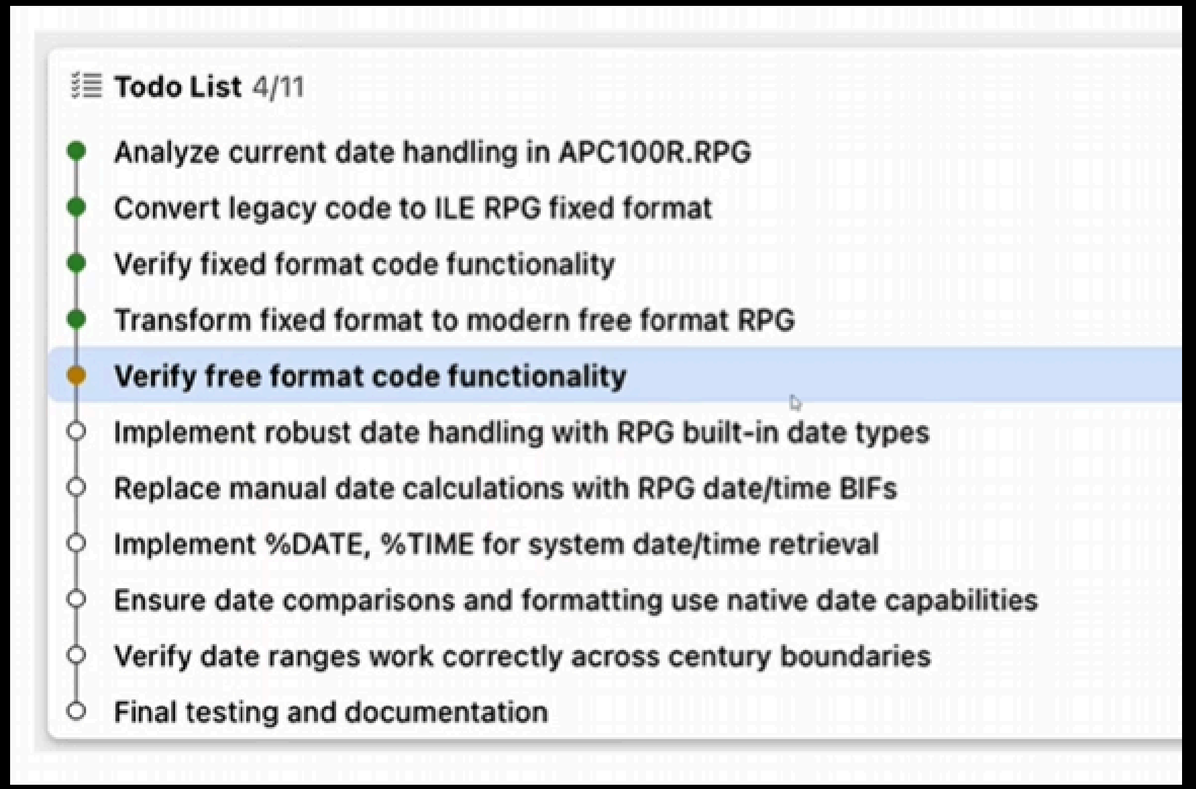
Much of the work is done through prompts.

Bob has built-in modes

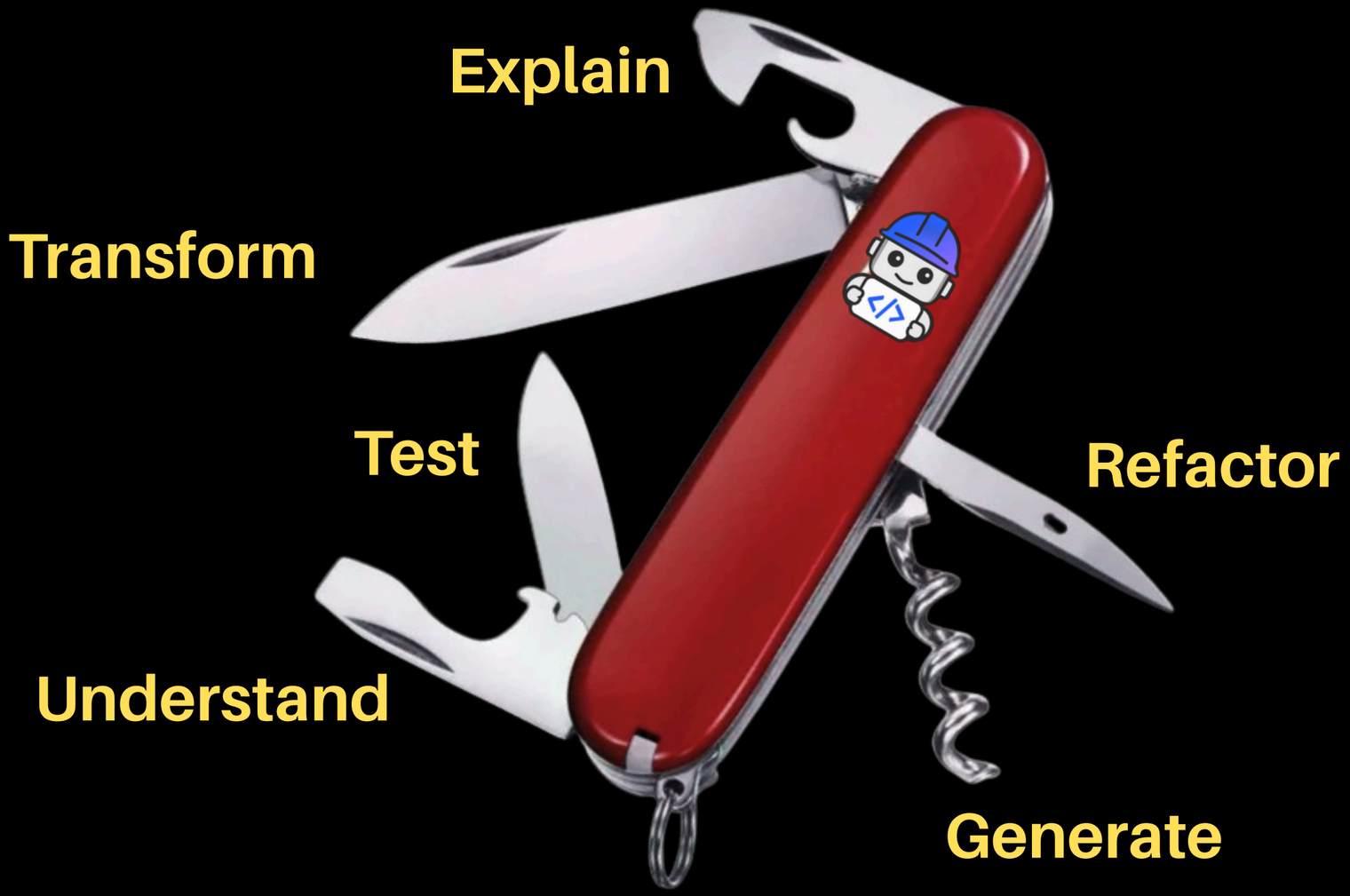


Bob has modes to do different tasks, such as **Ask** for general assistance, **Plan** for analysis and design and **Code** to create or modify code. You can create your own modes, too.

Whatever the task, Bob breaks it down into smaller pieces, creating an editable To Do List.



Bob has **six** main functions for developers ...



Explain

Simple: Explain a subroutine in a program



Hey Bob, explain the subroutine LOADSR in program ILE553

Moderate: Explain a full application

Explain the architecture of this application with front-end and back-end components. Provide details of how to test both. Include documentation with diagrams and append to context/architecture.md'.



Explain (Continued)

Complex: Explain a full 1980's ERP system(12 000 files, 1.5 million lines of code !)



Hey Bob, you're an RPG Expert. Using the code in the following codebases:

QCLSRC (CL), QDDSSRC (DDS), QPNLGRPSRC (Panel Group source files) and QRPGSRC (RPG source file containing main business logic)

Do the following:

- Identify major modules based on file naming patterns
- Analyse file types and their relationships
- Create high-level component-relation diagram
- Create internal diagrams for major modules
- Prepare README with all diagrams and explanations.

Explain (Continued)

Bob creates a To-do list, identifies modules, data flows, creates diagrams and documentation, as requested.

Todo List Created

- Identify major modules/domains based on file naming patterns
- Analyze file types (PF, LF, DSPF, PRTF) and their relationships
- Map program flows and data access patterns
- Create high-level component-relation diagram
- Create internal diagrams for major modules
- Prepare README with all diagrams and explanations

Now I'll start by identifying the major modules/domains based on file naming patterns the file names that suggest different modules:

1. AP - Accounts Payable (e.g., APC123R.RPG, APHEADP.PF, APE070R1.RPG)
2. AR - Accounts Receivable (e.g., AR030.RPG, AROPNL14.LF, ARR030ALL.RPG)
3. GL - General Ledger (e.g., GLA001.RPG, GLMASTWK.PF, GLP230R.RPG)
4. IN - Inventory (e.g., IN200A.RPG, IN201D00.DSPF)
5. PO - Purchase Orders (e.g., PODETL07.LF, POITEMJY.LF, POMASTJZ.LF)
6. PR - Payroll (e.g., PR001D04.DSPF, PR004.RPG)
7. WO - Work Orders (e.g., WOMAST.PF, WOMTRNP.PF)
8. BK - Banking (e.g., BKR215V.RPGLE, BKC020.clle)
9. CR - Credit (e.g., CR055.RPG, CRMNU2.DSPF)
10. HF - Human Factors/HR (e.g., HFM330D.DSPF, HFMNTRNP.PF)



This powerful functionality makes onboarding new team members easier than ever before!

Understand

Simple : Unfamiliar software.



Hey Bob, help me understand this application that has no documentation. What is the purpose of this software? How do I start this application?

Moderate: A 1960's legacy program that uses the RPG logic cycle

Help me understand how the RPG logic cycle loops through @/QSRC/TM100s.rpg36 program, explain how different sets of input specs are used. Create a flowchart of how the logic works and explain the idiosyncrasies of the language to someone who only understands javascript. Include how indicators KA to KY map to function keys and what purpose they serve. Diagrams will be helpful.



Refactor

Refactor variables



Rename all explicitly declared 6 character variable names to longer, meaningful, descriptive names.

Use modern date handling functionality

Convert legacy code into ILE Fixed format, then to ILE Free format. Use %built in functionality to handle dates.



Refactor Output specifications



Refactor the Output specifications in this program to create an externally described PRINTER file and a DISPLAY file.

Generate

So many use cases!

Code	Snippets, full programs, suites of programs, ENTIRE systems - theoretically in ANY language.
Database	All forms of structured Data (Files, Tables, views, indexes). turning unstructured data into structured (HUGE!!)
User Interfaces	Green screen' applications to modern front-ends (eg react, angular, flask, html)
Documentation	All kinds of documentation (comments, program and ERP level) All sorts of diagrams can be generated.
DevOps / Automation	Scripts, impact analysis reports, interpretation of graphs and data. Enormous scope for greater analysis in this area.







Transform Again, so many use cases !

Code	Transform languages into more modern versions. Monolithic to modular (RPG - using procedures, modules, service pgms).
Database	Structured Data into AI friendly SQL. Transform unstructured data into structured (huge potential to uncover treasure!).
User Interfaces	Transform Green screen applications into modern front-end applications (for example, basic HTML, CSS and Javascript.)
Documentation	Transform outdated documentation into something up-to-date at program, module or ERP level.
DevOps	Modernize scripts, improve CI/CD Pipelines.







Test

Create test data



Hey Bob, can you generate some test data for new files PF100 and PF101. Ten records each please.

Create a test plan

Explain the architecture of this application (front-end and back-end components). Create a full test plan of how to test both components. Include data flow diagrams in detailed documentation.

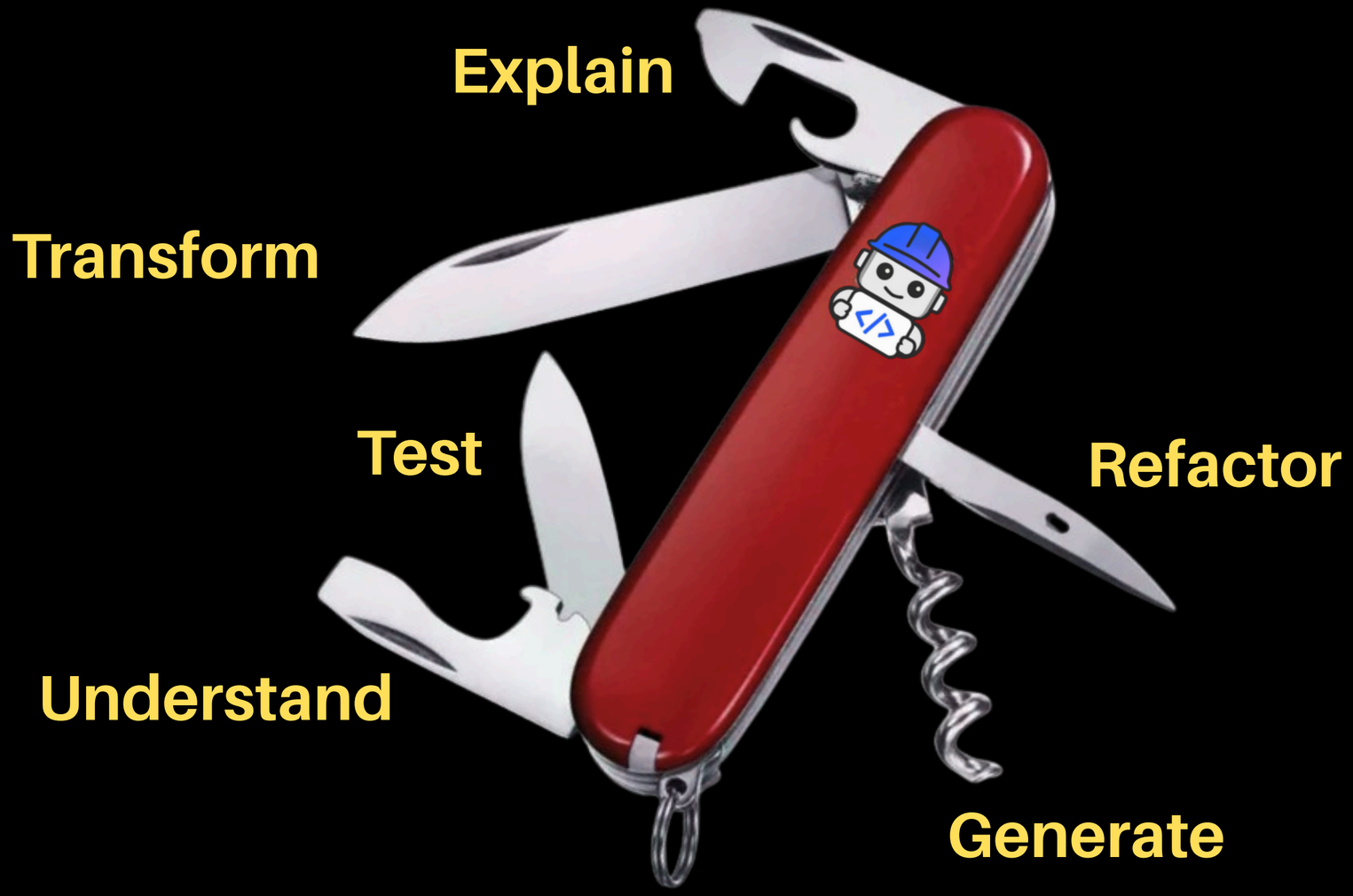


Leverage RPGUNIT



Write a set of unit tests across SNP125 module to test access to the database. Use the RPGUNIT framework and store the tests in a new file.

Reminder of the six main functions ....



Remember, we're **all** in the same boat, so just experiment and have fun!



Andrej Karpathy

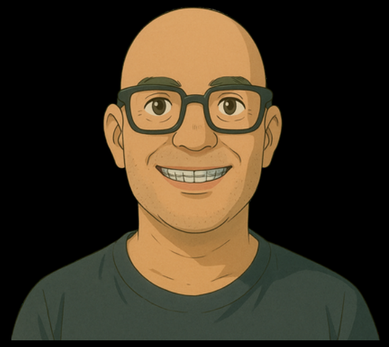
Co-founder, OpenAI
Former Director of AI, Tesla

Andrej Karpathy
@karpathy

I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts, contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.

4:36 AM · Dec 27, 2025 · **16.2M** Views

I hope you enjoyed the presentation



Neal Whittle

If there is any way I can help you buddy up with Bob, just let me know!